

Sreya Kalyan

March 4, 2024

CS 2160 – Mon/Wed (12:15pm)

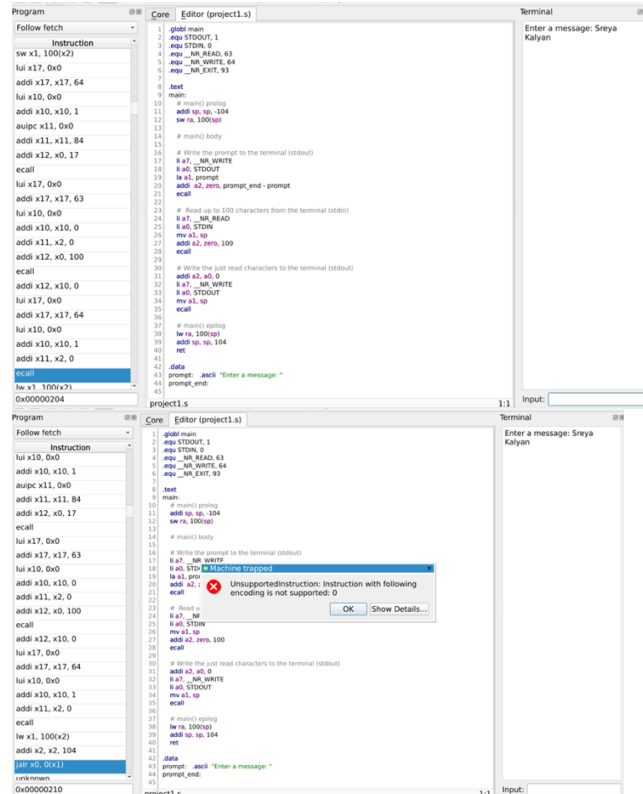
Prof. Keith Paarporn

Project 1 – Read Me Document

This project involved taking some code that was provided and refactoring it so that the functionality stayed the same but the code was a little more modular. Then, basing off of the results of the refactoring we did as well as some provided C-code, write a new programme that implements a certain set of functions and achieves the same result but through a different implementation with some required functions which are described with some base code to help us start.

Note: For the lib-c.c function, you must keep hitting “Run” because it prints and reads information literally character by character.

Part 1 - For Getting Started:



- I had no issues regarding this and was able to easily start working with the simulator and see all the different controls that you could manipulate.

Part 2 - For Refactoring Functions:

- For this, I tested with my name and was able to get it working correctly. I then tried with different iterations of my name as well as with special characters with and without letters to see if it would affect the code in any way and was able to get it working without errors. You first hit “Run,” and then when it pauses, it will prompt you to enter your name. Once you enter your name and hit “Run” again, it will consume your input, and then hitting “Run” after it pauses once more will make it print out your input to the terminal. Everything works fine.

The screenshot displays the RISC-V simulator interface. The main window is divided into three panes: Program, Editor, and Terminal.

Program Pane: Shows a list of instructions. The instruction `ecall` is highlighted in blue. Below the list, the address `0x0000022c` is shown.

Editor Pane: Displays the assembly code for `project1-redo.s`. The code includes prologue, body, and epilogue sections with various instructions like `lw`, `sw`, `addi`, `mv`, `li`, and `ret`. Comments explain the purpose of each instruction, such as restoring registers and saving stack space.

Terminal Pane: Shows the output of the program. It displays the prompt `Enter a message: Sreya` and the input `Enter a message:Sreya`. Below the terminal, there is an input field labeled `Input:`.

Messages Pane: Located at the bottom, it is currently empty.

File Machine Windows Options Help

1x 2x 5x 10x Unlimited Max

Program

Follow fetch

Instruction

auipc x10, 0x0

addi x10, x10, 165

jal x1, 0x238

addi x2, x2, -16

sw x10, 0(x2)

sw x11, 4(x2)

sw x12, 8(x2)

sw x17, 12(x2)

addi x12, x11, 0

addi x11, x10, 0

lui x10, 0x0

addi x10, x10, 1

lui x17, 0x0

addi x17, x17, 64

ecall

lw x10, 0(x2)

0x0000022c

Core Editor (project1-redo.s)

```

39 # Epilog
40 lw a0, 0(sp) # Restore a0
41 lw a1, 4(sp) # Restore a1
42 lw a2, 8(sp) # Restore a2
43 lw a7, 12(sp) # Restore a7
44 addi sp, sp, 16 # Restore stack pointer
45 ret
46
47 read_string:
48 # Prolog
49 addi sp, sp, -12 # Make room for 3 items on the stack
50 sw a1, 0(sp) # Save a1
51 sw a2, 4(sp) # Save a2
52 sw a7, 8(sp) # Save a7
53 # Body
54 mv a2, a1 # Move a1 into a2
55 mv a0, a1 # Move a0 into a1
56 li a0, 0 # Put STDIN code into a0
57 li a7, 63 # Put NR_READ code into a7
58 ecall
59 # Epilog
60 lw a1, 0(sp) # Restore a1
61 lw a2, 4(sp) # Restore a2
62 lw a7, 8(sp) # Restore a7
63 addi sp, sp, 12 # Restore stack pointer
64 ret
65
66 .data
67 prompt: .ascii "Enter a message: "
68 buf: .space 100
69 buf_end:
70
71 project1-redo.s

```

Terminal

Enter a message: Sreya
Enter a message:Sreya
Enter a message:!!Sreya!
Input:

Messages

File Machine Windows Options Help

1x 2x 5x 10x Unlimited Max

Program

Follow fetch

Instruction

auipc x10, 0x0

addi x10, x10, 165

jal x1, 0x238

addi x2, x2, -16

sw x10, 0(x2)

sw x11, 4(x2)

sw x12, 8(x2)

sw x17, 12(x2)

addi x12, x11, 0

addi x11, x10, 0

lui x10, 0x0

addi x10, x10, 1

lui x17, 0x0

addi x17, x17, 64

ecall

lw x10, 0(x2)

0x0000022c

Core Editor (project1-redo.s)

```

39 # Epilog
40 lw a0, 0(sp) # Restore a0
41 lw a1, 4(sp) # Restore a1
42 lw a2, 8(sp) # Restore a2
43 lw a7, 12(sp) # Restore a7
44 addi sp, sp, 16 # Restore stack pointer
45 ret
46
47 read_string:
48 # Prolog
49 addi sp, sp, -12 # Make room for 3 items on the stack
50 sw a1, 0(sp) # Save a1
51 sw a2, 4(sp) # Save a2
52 sw a7, 8(sp) # Save a7
53 # Body
54 mv a2, a1 # Move a1 into a2
55 mv a0, a1 # Move a0 into a1
56 li a0, 0 # Put STDIN code into a0
57 li a7, 63 # Put NR_READ code into a7
58 ecall
59 # Epilog
60 lw a1, 0(sp) # Restore a1
61 lw a2, 4(sp) # Restore a2
62 lw a7, 8(sp) # Restore a7
63 addi sp, sp, 12 # Restore stack pointer
64 ret
65
66 .data
67 prompt: .ascii "Enter a message: "
68 buf: .space 100
69 buf_end:
70
71 project1-redo.s

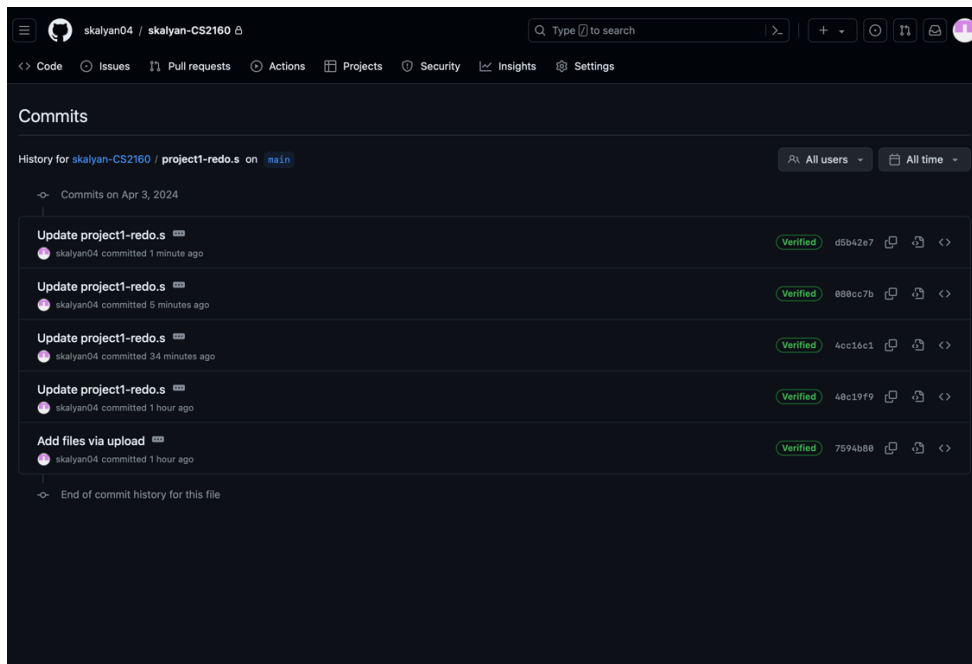
```

Terminal

Enter a message: Sreya
Enter a message:Sreya
Enter a message:!!Sreya!
Enter a message:?!
Input:

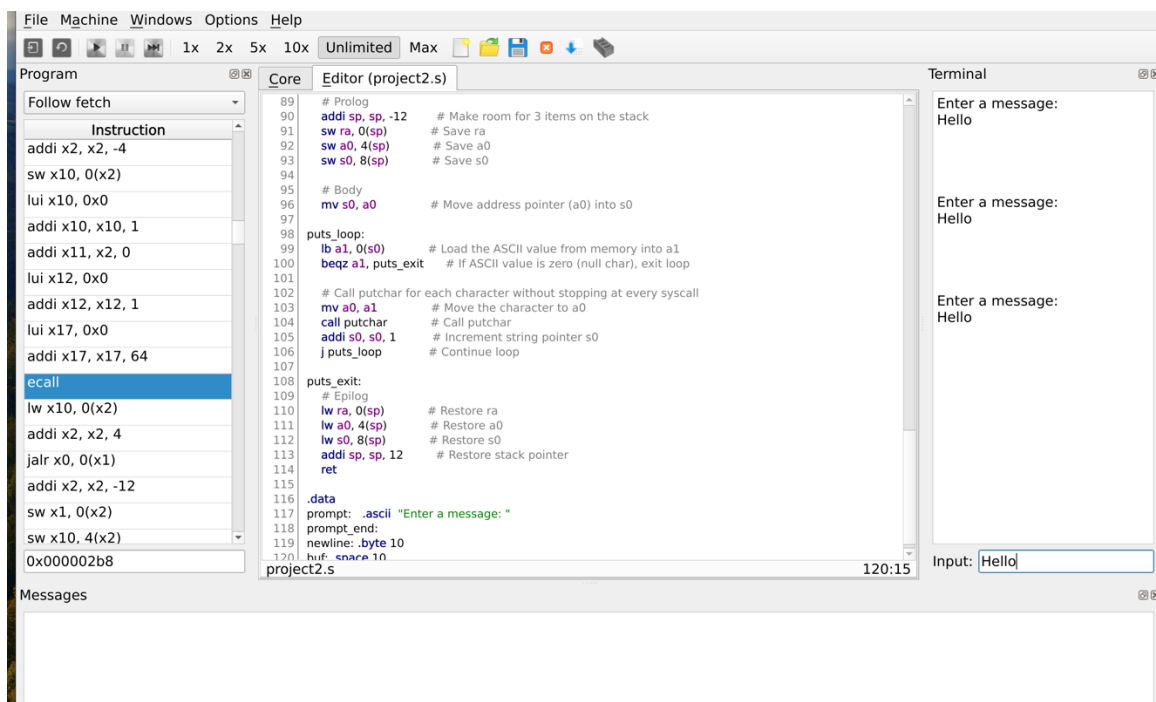
Messages

- In regards to my logging, I was logging after every function that I was able to write. So I first wrote the read_string function, logged it, wrote the write_function, logged it, and then made separate adjustments to remove and add code. All of it was done in separate logs and then made the final adjustments pertaining to RISC-V assembly coding standards and practices as well as comments and spaces.

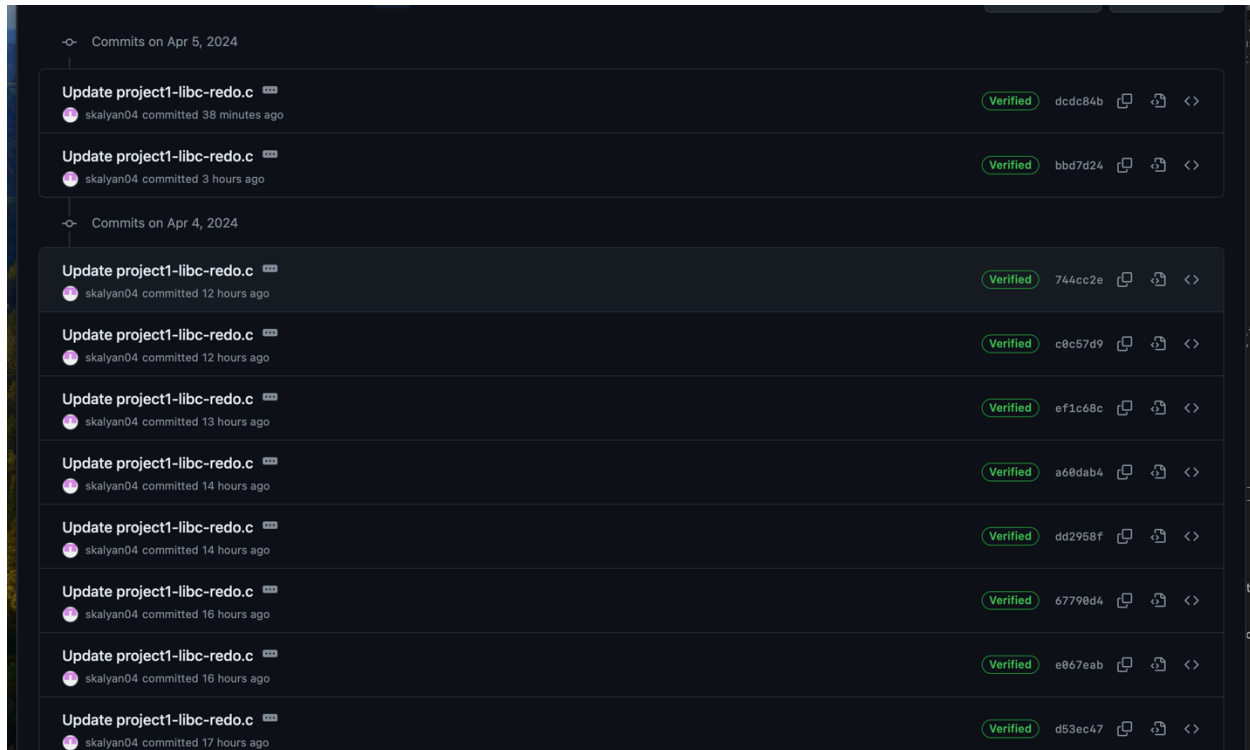


Part 3 – For libc:

- I was able to make sure that this does not crash, and it does work. However, it doesn't run very well. For this I tested for many inputs but I encountered the same issues involving user input as well as continuing to hit "Run" to make it work.



- The code requires that you must keep hitting the “Run” button because it will print out each character one by one. The same goes for reading in the user input and then printing it back out to the console. The code requires that you must constantly hit the “Run” button so that it will function. It isn’t the most effective way at all.



- I was logging quite often because I was making so many adjustments to make it work and at least run instead of crashing or not even compiling. I was compiling with every minute change I made and was make sure to be as careful as possible so that I could always go back and retrieve any code in the case that something I was trying or experimenting with would backfire.